TLM Creation

Rapid Training

X-2025.06

Module 1: Lab A

Create the Model Skeleton of an Interrupt Timer

© 2025 Synopsys, Inc.

Table of Contents

Lab Overview *2*

Task-1: Create a Tracking Feature List for the Timer Model **2**

Task-2: Create a TLM Model from an "*empty*" template **3**

Task-3: Create the Shell of the Timer Model **4**

Task-4: Specifying the Interface of the Timer Model **6**

Task-5: Specifying the Memory Elements of the Timer Model **8**

Task-6: Adding Behavior to the Memory Elements **11**

Task-7: Exploring the Parameters page **13**

Task-8: Save the Model, Generate Code, and Build **13**

Task-9: Examine the Generated Infrastructure and Files **15**

Task-10: Build and Run Default Unit Tests **18**

Task-11: What We Have Learned **20**

Task-12: What's Next **20**

Lab Overview

Your task for this exercise is to use Virtualizer Studio's TLM Creation perspective to create a TLM model of a Programmable Interval Timer (PIT) component. The specification for the PIT model example is provided in the Training Programmable Interval Timer Specification document, that you should read first.

Before you start with the actual model implementation, you should enumerate all behavior, and create a feature list for tracking the testing and implementation of these features, as described in the *Synopsys VDK Creation Methodology*. A sample of such a tracking sheet is provided in this training package as well.

In this lab, you will create the shell of the PIT timer model, from an *empty* template in the TLM Creation perspective of Virtualizer Studio as a starting point. For the specification of the memory elements (register files, register, bit-fields), you can optionally use an IP-XACT file to import these definitions into your TLMC project.

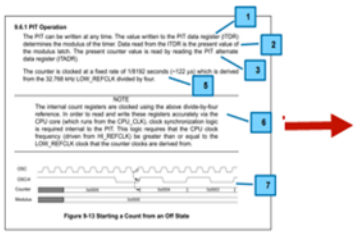
This introductory lab will also take you through the infrastructure of the tool such as the context specific help system, the generated directories, and files for a TLMC project, the basics of the unit-testing environment.

Task-1: Create a Tracking Feature List for the Timer Model

- ***Motivation**: This step calls for a breakdown of the model description into a list of features. In the implementation phase, you need to create a unit test for every listed feature and add an implementation. When the unit test starts to pass, you can mark it against the feature in the tracking sheet itself. This makes the implementation of the model much smoother and ensures that all the features are working as expected.*

Following tips can help you while creating the features-list tracker of the model to be created.

- Read through every chapter and enumerate all pieces of information that might need implementation.
- List up all items into Excel - one sentence per item.
- If unclear, add a note or question and your assumptions in the appropriate columns.



SoC Specification
(SW User Manual)

	impl	tested	Function Description	Questions
			9.6.1 Interval Timer, Mode of Operation	
2			writing to ITDR sets the modulus of timer	
3			reading from ITDR shows modulus	
4			reading from ITDR shows currrent counter value	
5	x		fixed clock from LOW_REFCLK/4 = 32.768/4	
6	x		clock syncronization logic	
7	x	x	start from off state	
8	x	x	2 modes (count down or free runing)	
10			modulus latch	
11			16-bit counter	
			9.6.2 Set-and forget	
1			operation mode is selected when RLD bit is set to 1	
2			when counted to zero then ITIF flag is set	
3			if zero (and in set and forget) then the modulus is loaded back into timer	
4			if the ITIE is set then interrupt pin is fired	when is the interu
5			can be initialized if OVW bit is set	
			9.6.3 Free Runing	
1			operation mode is selected when RLD bit is set to 0	
2			rollover from 0x0 to 0xFFFF, no use of modulus latch	
3			when counted to zero then ITIF flag is set	
4			if the ITIE is set then interrupt pin is fired	
5			can be initialized if OVW bit is set	

Tracking List

For viewing a sample tracker file, see TimerFunctionalityTracking.xlsx in the training data package. For more details on the Timer model, refer to the Training Programmable Interval Timer Model Specification document.

Task-2: Create a TLM Model from an "*empty*" template

- ***Motivation**: By using the model template while creating a new tlmc project, several predefined items such as interfaces, registers, etc. are added to the model and can be readily used. If needed, you can edit or remove them as per your requirements.*

1. Start Virtualizer Studio

   Open a console and invoke Virtualizer studio from the Synopsys Virtual Prototyping installation.

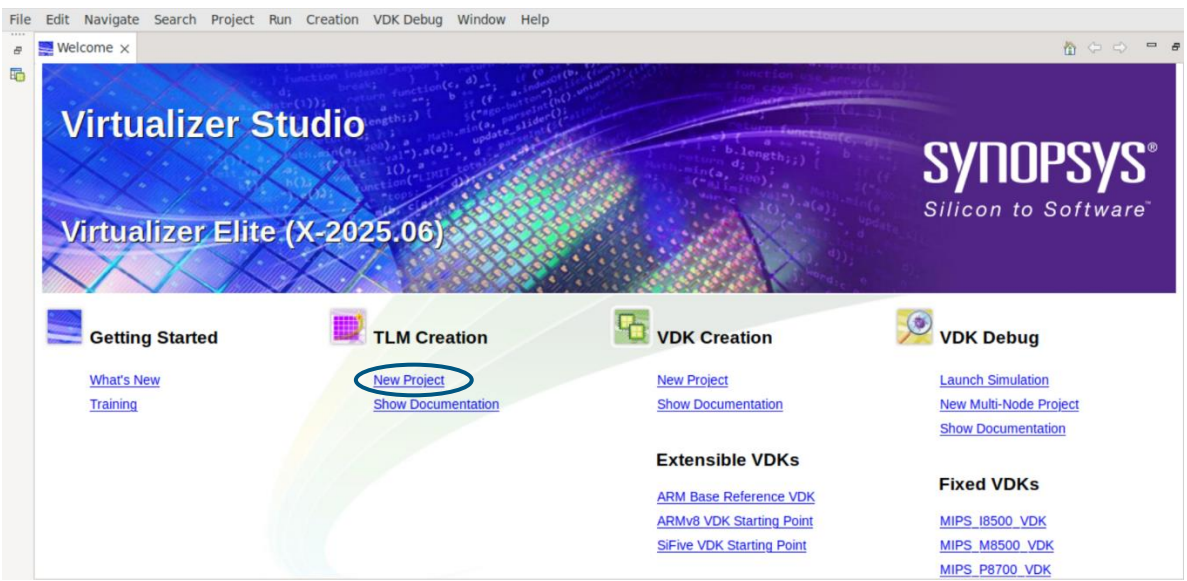
   - `% source <installDir>/SLS/windows/setup.bat vi[rtualizer] (Windows)`

     `% source <installDir>/SLS/linux/setup.csh|sh vi[rtualizer] (Linux)`

   This sourcing option picks up the Virtualizer *Elite* or *Base* license in that order, whichever is available. For more details on other options, refer to the VP documentation.

   - `% vs&`

   - `% cd <TrainingData>/module_1`



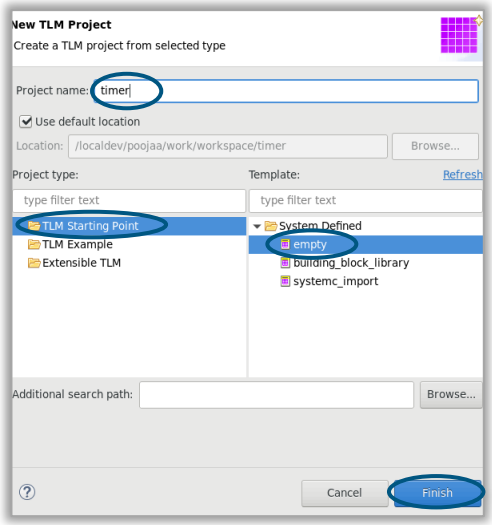
   The tool opens and displays the Workspace Launcher window.

2. Specify your workspace location, where you want to save your projects and click ***OK*** on the Workspace Launcher window.

   Note: Workspace is where Virtualizer Studio will store all your model and VDK projects and associated files.

   When the Welcome screen appears, click ***New Project*** under ***TLM Creation***.

3. Specify the following in the ***New TLM Project*** dialog:



   - Type `timer` in the ***Project name*** field,
   - Select the `TLM Starting Point` option in the ***Project type*** field,
   - Select the `empty` Model Template from the available options.
   - Click ***Finish***.

   When prompted, click ***Open Perspective*** to open the ***TLM Creation*** perspective.

   The ***TLM Creation*** perspective appears. We will examine the information displayed in different tabs of this perspective in the subsequent sections.

Task-3: Create the Shell of the Timer Model

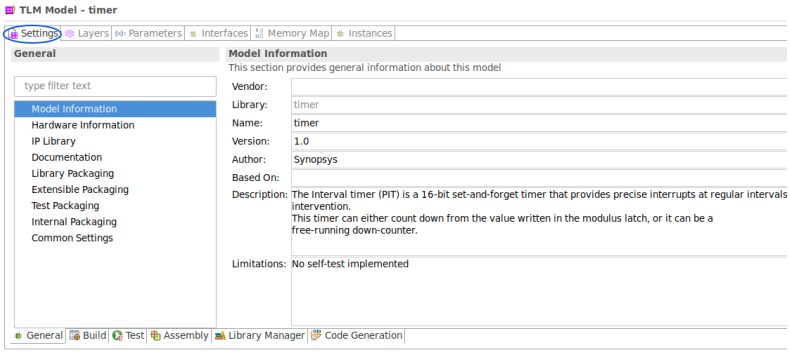
- ***Motivation***: *Now that we have created a base for our timer TLM model, we would be refining its components and filling the missing gaps. This task is divided in the following heads:*
  - *Filling in the Generic information about the model*
  - *Specifying Interfaces details*
  - *Specifying Register details*
  - *Binding register/memory to the corresponding interface*

Let's start with exploring the information displayed for the timer model in the ***Settings*** tab.

4. Click to open the ***Settings*** tab of the model editor and focus on the ***General*** page.

   As we selected an empty template, there will be minimal to no auto-populated information in this view. Let's add the requisite information to our model as follows!

   i. In *Model Information* section, specify the following details:



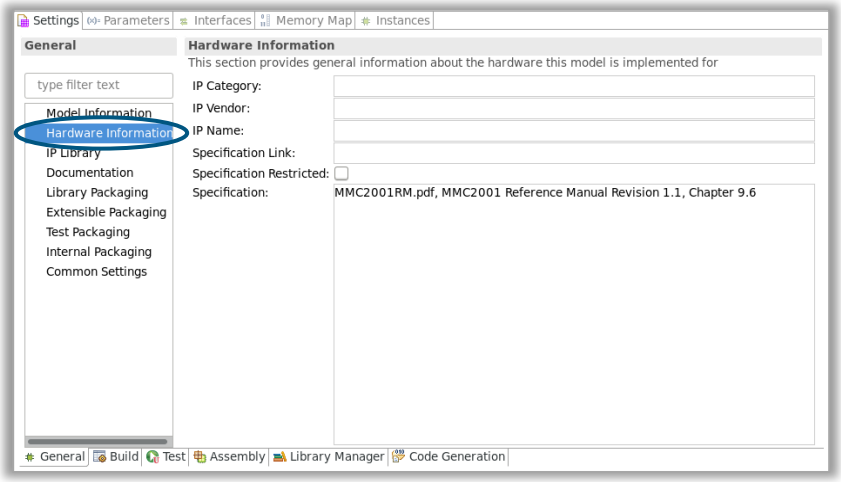
   - ***Model name***: Field is populated from the project name used.
   - ***Version***: Specify `1.0`. This information later reflects in the FastBuild library,
   - ***Author***: Specify `Synopsys` in this field,

- ***Description***: *The Interval timer (PIT) is a 16-bit set-and-forget timer that provides precise interrupts at regular intervals with minimal processor intervention.*
  This timer can either count down from the value written in the modulus latch, or it can be a free-running down-counter.

- ***Limitations***: No self-test implemented

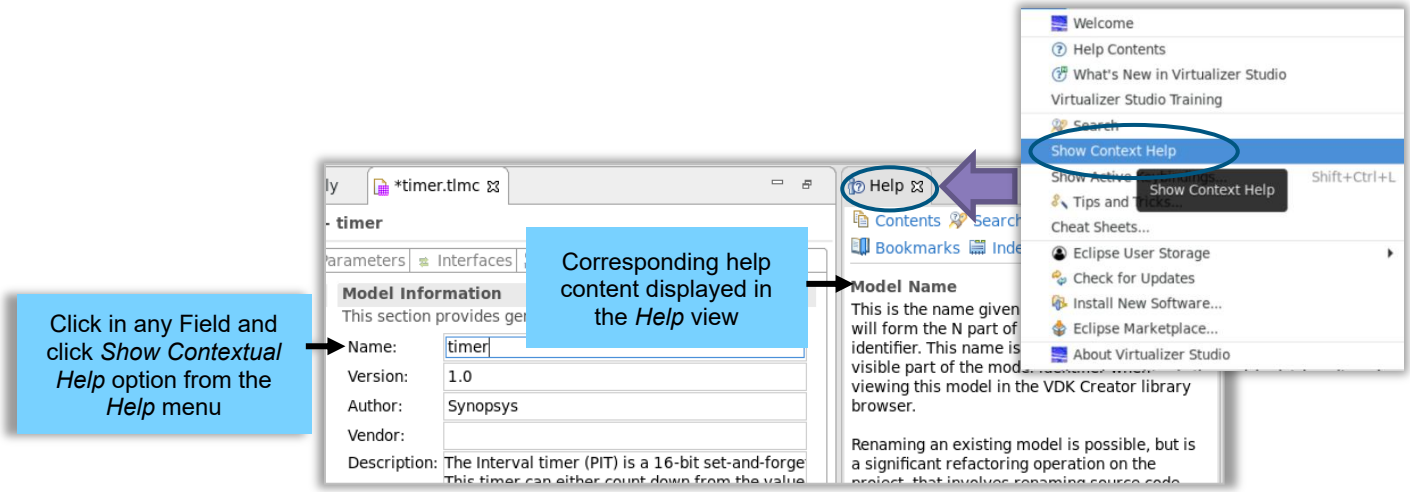
ii. In *Hardware Information* section, specify the following details:

- ***Specification***: *MMC2001RM.pdf, MMC2001 Reference Manual Revision 1.1, Chapter 9.6*



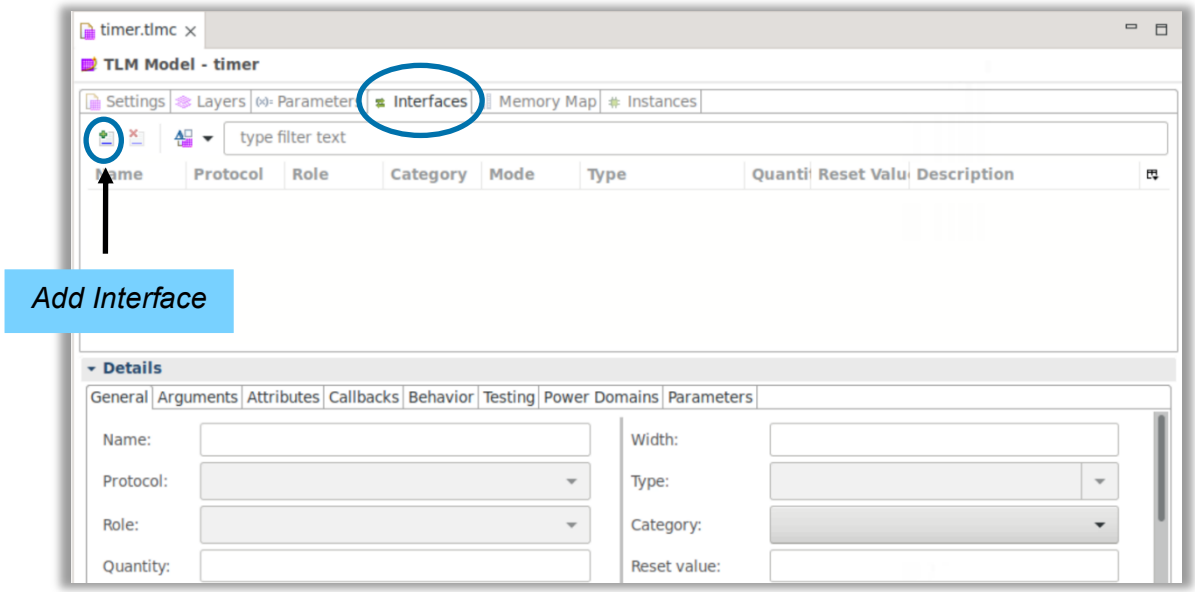
iii. Save your project using the ***Save All*** button from the tool bar.

<u>Tip:</u> The ***TLM Creation*** perspective of Virtualizer Studio provides an exhaustive context sensitive help for each element of the GUI. To view this tab in case it is not displayed, click ***Show Contextual Help*** option from the ***Help*** menu. It is recommended, at least until you are familiar with all GUI and library items, to leave this contextual help open.



Task-4: Specifying the Interface of the Timer Model

- ***Motivation**: The **Interfaces** page displays the ports and sockets through which your model interfaces with other models in the VDK design. Since we are starting with an empty template, you would need to add the interfaces as per the requirements.*
  We need the following interfaces for the timer model:
  -a Memory target socket,
  -a Clock input,
  -a Reset input,
  -an Interrupt output pin.



5. Click to open the *Interfaces* tab of the model editor and click on the ***Add Interface*** button to add a new element.

   Add the required interfaces as follows.

   i. Memory target socket with the following details.

      - Name: `p_registers`
      - Protocol: `TLM2`
      - Role: `target`
      - Category: `Memory`
      - Mode: `LT`
      - Type: `scml2::ft_target_socket<32>`
      - Description: Memory target socket

Note: You just need to specify the ***Protocol*** and ***Role*** for the interface entry, and the tool automatically determines respective ***Category*** and ***Type***.

ii. Clock input with the following details.

- Name: `p_clk`
- Protocol: `Clock`
- Role: `input`
- Category: `Clock`
- Mode: `n/a`
- Type: `sc_core::sc_in<bool>`
- Description: *Clock Input*

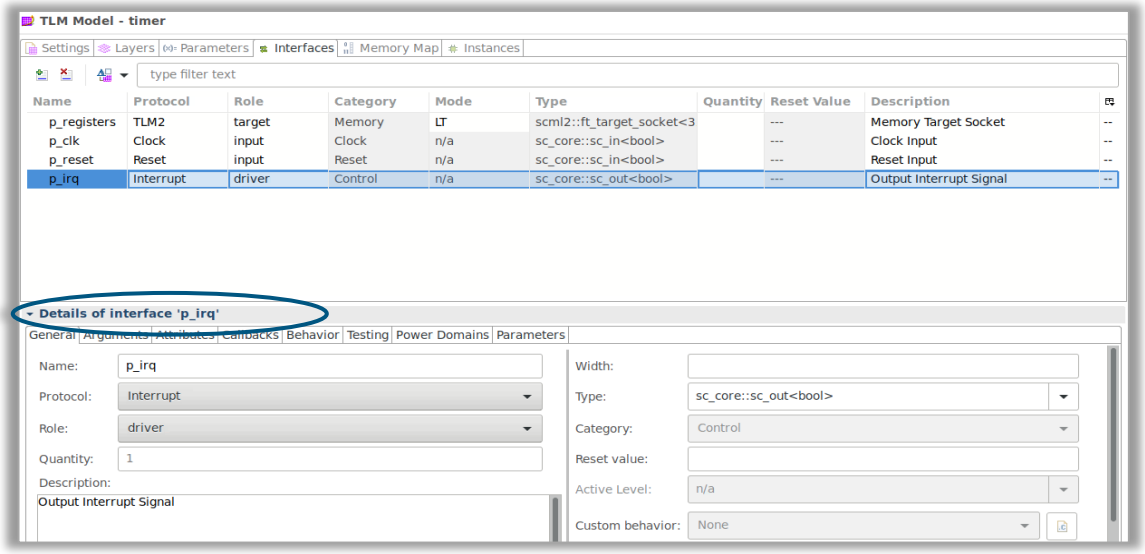
iii. Reset input with the following details.

- Name: `p_reset`
- Protocol: `Reset`
- Role: `input`
- Category: `Reset`
- Mode: `n/a`
- Type: `sc_core::sc_in<bool>`
- Description: *Reset Input*

iv. Interrupt output pin with the following details.

- Name: `p_irq`
- Protocol: `Interrupt`
- Role: `driver`
- Category: `Control`
- Mode: `n/a`
- Type: `sc_core::sc_out<bool>`
- Description: Output Interrupt Signal

Tip: The window also displays all the interfaces in a tabular format and a ***Details*** pane, with more information. You can use either of the two to update any information. However, the advantage of editing the table directly, is that you do not need to use your mouse, and can easily browse the table cells using keyboard shortcuts i.e. *Tab* and *Arrow keys* to traverse and *Enter* key to confirm your changes



Task-5: Specifying the Memory Elements of the Timer Model

- ***Motivation****: The* ***Memory Map*** *page displays the model's memories, register files, registers, and their bit fields. You would need to add the memory elements as per the requirements in the* ***Memory Map*** *page as described in the Task-5 a.*
  If you have an IP-XACT specification of your model containing its memory map details, you can import it directly to have the elements added automatically in your TLM Creator project. This alternative flow is described in Task-5 b.

Task-5 a: Entering the Memory Elements in the Memory Map Page

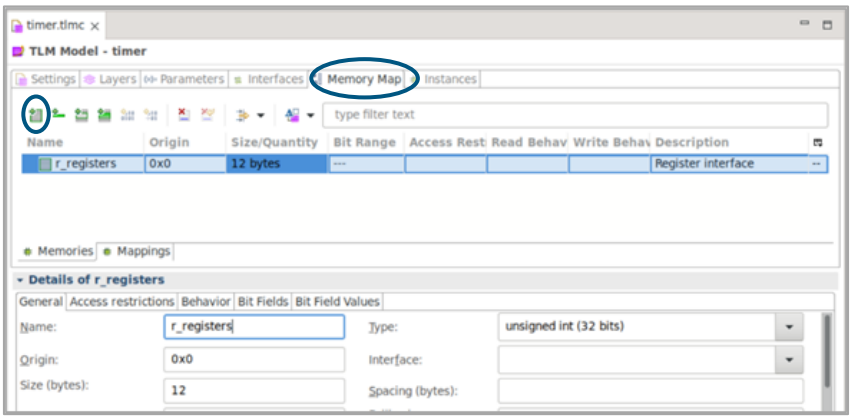
6. Click to open the ***Memory Map*** tab and add a new memory as *m_regs* using the following steps.

   i. Click the ***Add Memory*** button,

   ii. Specify the Name as *r_registers*,

   iii. set ***Size/Quantity*** to `12 bytes`, and

   iv. set ***Description*** as `Register interface`.



Tip: *Keyboard navigation*: Use the Arrow-keys to navigate all the elements in the table. In edit mode, you can navigate using `<Ctrl-Tab>` and `<Ctrl-Shift-Tab>` to move up and down in the column.

Keyboard shortcuts: Refer to the following illustration for some useful shortcuts for this UI.

Add Bitfield	Ctrl+N Ctrl+B
Add Child Bitfield	Ctrl+N Ctrl+C
Add Memory	Ctrl+N Ctrl+M
Add Memory Alias	Ctrl+N Ctrl+A
Add Register	Ctrl+N Ctrl+R
Add Register Group	Ctrl+N Ctrl+G

Next, lets add registers under the `r_registers` memory.

i. Select the `r_registers` entry and click the ***Add Register*** ( ) button. Alternatively, use the keyboard shortcut `<CTRL-n><CTRL-r>`, or right click ***Add Register*** from the context menu,

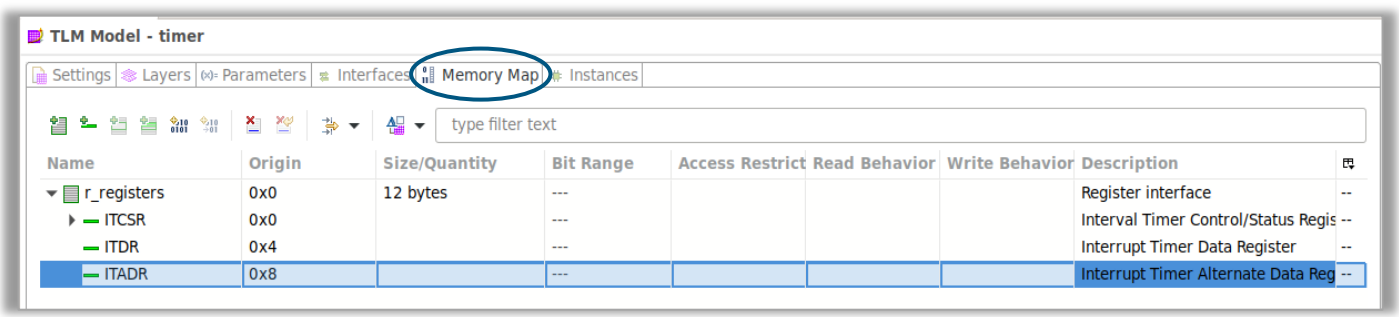
ii. Specify the ***Name*** as *ITCSR,* and set ***Description*** as `Interval Timer Control/Status Register`.

iii. Next, add a register element again using `<CTRL-n><CTRL-r>` and set the ***Name*** as *ITDR,* ***Origin*** as `0x4`, ***Description*** as `Interrupt Timer Data Register`.

iv. Similarly, add an *ITADR* register with ***Origin*** at `0x8`, and ***Description*** field set as `Interrupt Timer Alternate Data Register`.

Alternatively, you can copy the *ITDR* register row and paste it using the context menu option, or simply use keyboard shortcuts, `<Ctrl+C><Ctrl+V>` and update the relevant details such as name and description accordingly.

Here's an updated table with all the registers added for the model.



Add Bit fields

The timer status/control register *ITCSR* requires the following defined bit fields.

Name	Position	Description
EN	0	Enable
RLD	1	Counter Reload Control
ITIF	2	Interrupt Timer Interrupt Flag
ITIE	3	Interrupt Timer Interrupt Enable
OVW	4	Counter Overwrite Enable
DBG	5	Debug Mode Control
DOZE	6	Doze Mode Control

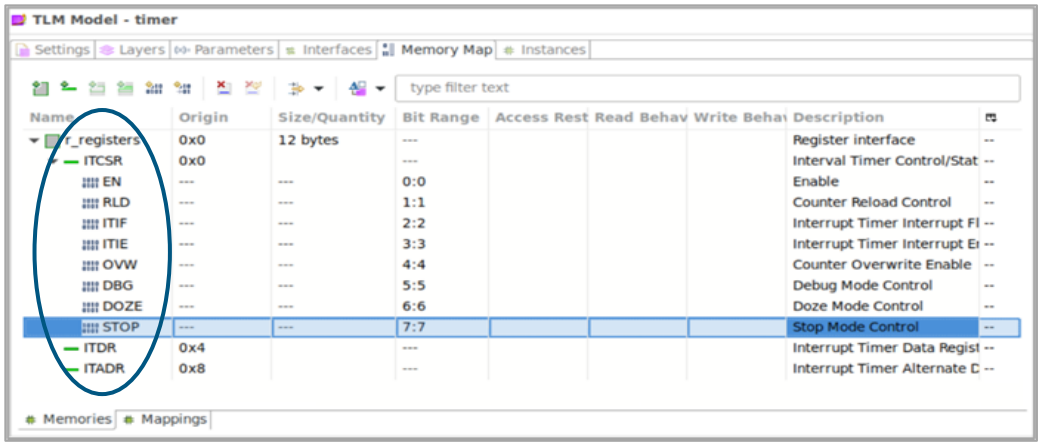
STOP	7	Stop Mode Control

Let's get on to define these bit-fields for the *ITCSR* register.

7. Select the *ITCSR* register and add a bit field by pressing the ***Add Bit Field*** button ( ) or by selecting ***Add Bitfield*** from the context menu, or by typing `<CTRL-n><CTRL-b>`.

    i. Rename the new bit field *bf* (which comes by default) to *EN* and set ***Bit Range*** as `0` and update ***Description*** field from the table accordingly.

    ii. Similarly, add the remaining bit-fields or use ***Copy/Paste*** options from the context menu (use keyboard shortcuts) to add them. Edit the ***Name***, ***Bit Range*** and ***Description*** for all the bit-fields as per the requirement table.



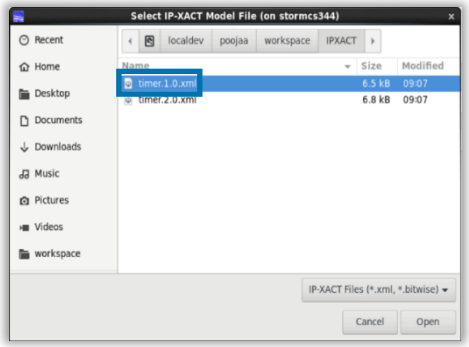
Task-5 b: Importing the Memory Map from an IP-XACT Specification

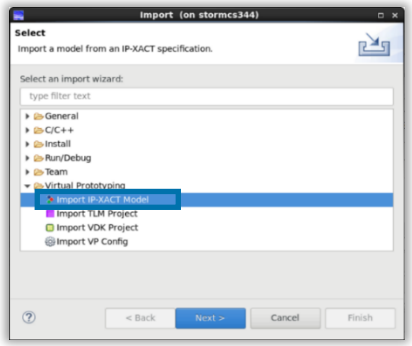
Note: If you have already completed *Task-5 a,* you can skip this task and continue with the Task-6. If you have done Task-5 a, but you still want to try out the IP-XACT import flow as well, simply remove the entire register list from your ***Memory Map*** page and continue with this task.

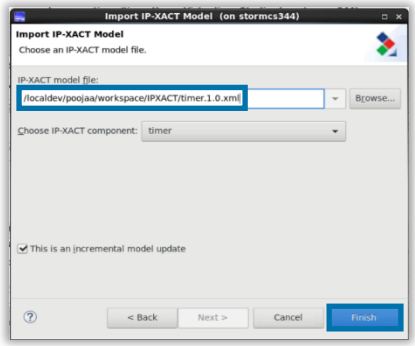
8. For the IP-XACT based flow for creating a model, first remove any existing memory element from the Memory Map table: select the memory element to be removed (e.g. `p_registers`) and click the ***Remove*** button or select the ***Remove*** option from the context menu.

9. Next, click ***File > Import …*** in the top-level menu and in the popping up ***Import*** dialog select ***Virtual Prototyping > Import IP-XACT Model*** and click the ***Next*** button.

10. In the next ***Import IP-XACT Model*** dialog, hit the ***Browse…*** button right to the ***IP-XACT model file*** field, navigate to the `…/module_1/IPXACT` folder and select the `timer.1.0.xml` file.







11. Now the ***IP-XACT model file*** field in the ***Import IP-XACT Model*** dialog is filled in, and the `timer` component that is contained in the IP-XACT model file is correctly recognized and displayed.

12. Click the ***Finish*** button to start the import.

    The import process starts and the issues occurring during the import process are displayed in the ***Tasks & Problems*** tab of Virtualizer Studio.

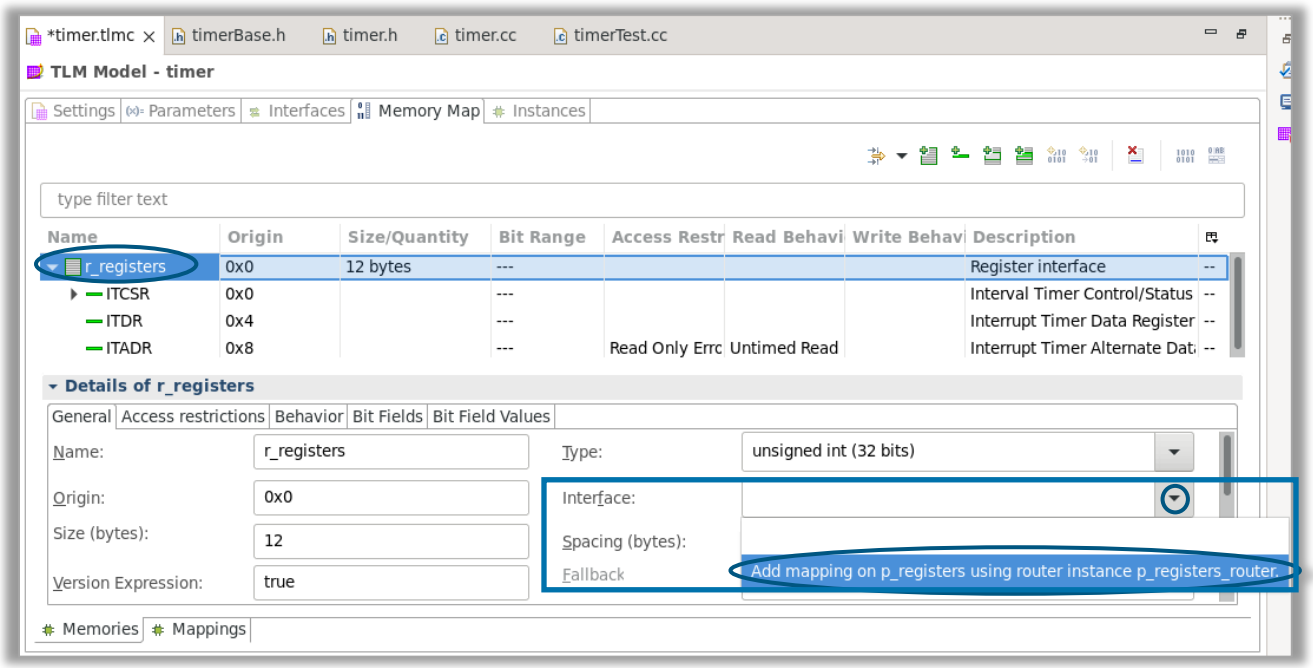
13. Now, switch to the ***Memory Map*** page of the Model editor to view the memory elements that have been imported from the IP-XACT model file.

Task-6: Adding Behavior to the Memory Elements

- ***Motivation**: The **Memory Map** page also allows you to specify the behavior of the memory elements. Here you can for e.g. specify callbacks on register accesses, access restrictions, etc.*
  In this lab we will just add some very basic register behavior – the rest is done in the following labs when we add more behavior to the model.

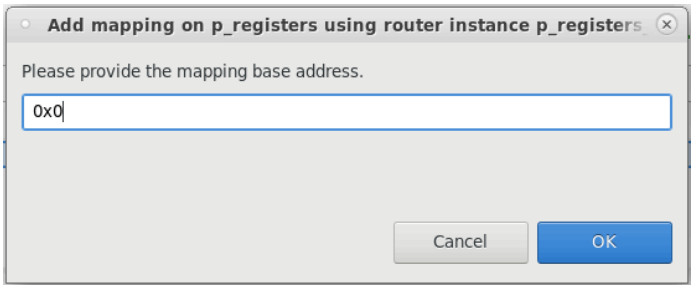
14. First, we need to make sure the register/memory is bound correctly to the interface in the ***Memory Map*** tab.

    i. Select the `r_registers` entry and expand the ***Details of r_register*** pane.



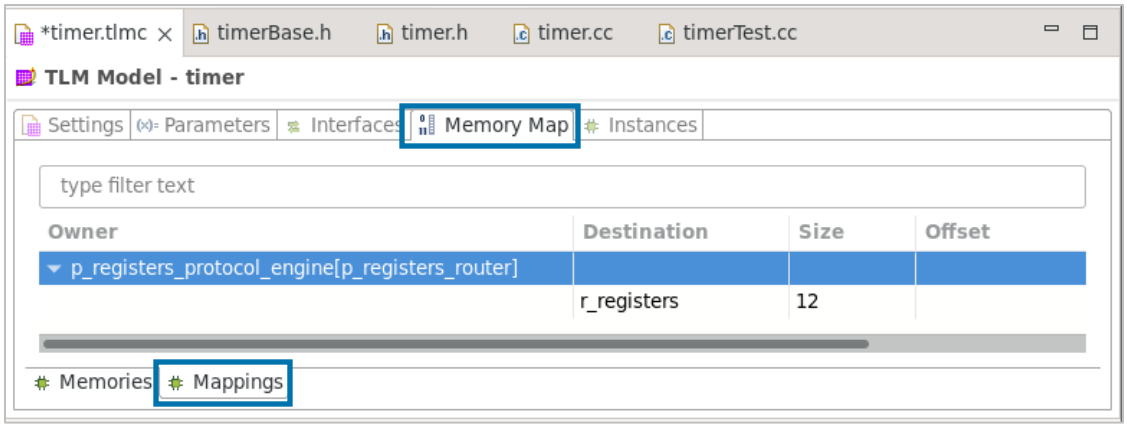
    ii. Click on the arrow to check the options and select the ***Add mapping on p_registers using router instance p_register_router…*** in the ***Interface*** field.

    Note: The same binding can be done from the ***Registers*** table directly also. Click on the ***Customize Table*** icon ( ) on the right-hand side of the table header and select the checkbox for ***Interface*** to add an ***Interface*** column to the memory table.



    iii. Provide the base address in the popup dialog and click ***OK***.

    iv. Click to switch to the ***Mappings*** page from the bottom of the Memory Map view.



The newly added mapping entry is displayed in the ***Mappings*** tab.

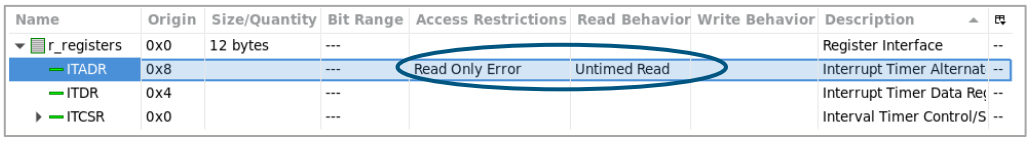
Next, you need to define some basic register behavior, i.e. what should happen on a register write or read. For this, you need to add callback routines that are called when a register is written or read.

These callback routines can be attached to registers as well as to individual bit-fields. However, you should avoid bitfield callbacks for implementing the functionality that depends on the results of other callbacks attached to the same register. You should rather use register callbacks in such cases.

You may use some of the predefined behaviors callback routines for read/write, or you can select from the provided ***Custom*** behaviors, and then edit/add relevant code to the routines in the source files. Refer to the dynamic help in Virtualizer Studio for more details and setting the right behavior for the callbacks.

15. The *ITADR* is a read-only register on the other hand, so we will only be adding ***Read Behavior*** for it, and on top of that add an ***Access Restriction*** to prevent (or help debug) illegal write accesses to the register.

    Select the *ITADR* register row, double-click on the ***Read Behavior*** field and specify `Untimed Read` as behavior type and leave the ***Write Behavior*** field empty. Also set the ***Access Restriction*** field as `Read Only Error` to ensure that it is a read-only register. So, if someone tries to write to it through the bus interface, the simulation will issue a runtime error, which would help to identify the root cause of the problem.



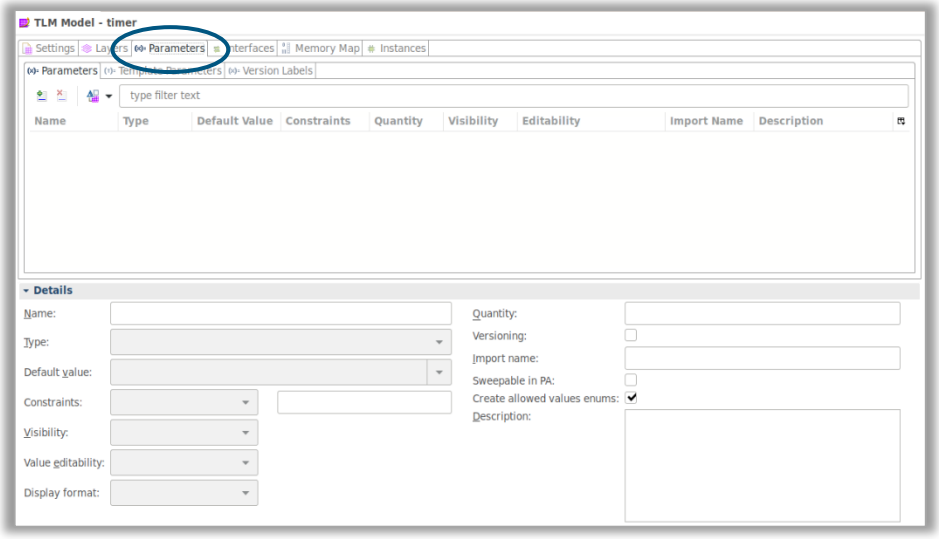
Name	Origin	Size/Quantity	Bit Range	Access Restrictions	Read Behavior	Write Behavior	Description	
r_registers	0x0	12 bytes	---				Register Interface	--
ITADR	0x8		---	Read Only Error	Untimed Read		Interrupt Timer Alternat	--
ITDR	0x4		---				Interrupt Timer Data Re	--
ITCSR	0x0		---				Interval Timer Control/S	--

<u>Note:</u> At this point, we skip setting other fields for the registers such as ***Reset value*** as those will be set in the behavior implementation step (covered in the subsequent Labs).

Task-7: Exploring the Parameters page

- ***Motivation**: The **Parameters** page displays the model parameters corresponding to the SCML parameters, which are added to the model source code by the code generator. These can be assigned a value at the simulation initialization time in the VDK authoring tools (VDK Creation perspective of Virtualizer Studio and Platform Creator).*

You can use the ***Table*** or ***Details*** section, for each parameter to define e.g. its ***Name***, ***Type***, and ***Default Value***. It is also possible to define constraints for a parameter either as a range or as a set of allowed values. Finally, the ***Parameters*** page allows you to define the visibility and editability properties of the parameter.



There are no parameters defined for the *timer* model in the requirements, so we will skip this tab for now.

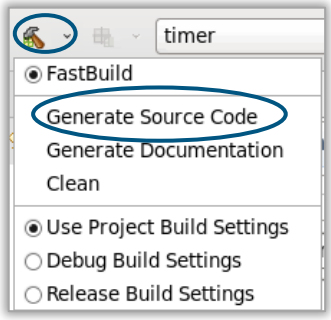
We have now completed the model definition and callbacks for the behavior implementation. We will be adding the behavior implementation in the subsequent labs. In this lab, now we will be running the default unit tests to see whether all code is consistent and without any errors.

Task-8: Save the Model, Generate Code, and Build

- *Motivation: After making all the changes, we can check the model for remaining issues and then save the model project, and build it to generate the source code, default implementation and unit tests.*

16. Click the ***Save*** option under the ***File*** menu to save the project.

    Check the ***Tasks & Problems*** tab. The tool does a semantic check and reports any issue or inconsistency, if found.



17. Click on the arrow next to the ***Build*** button on the toolbar and select ***Generate Source Code*** option to generate the code.

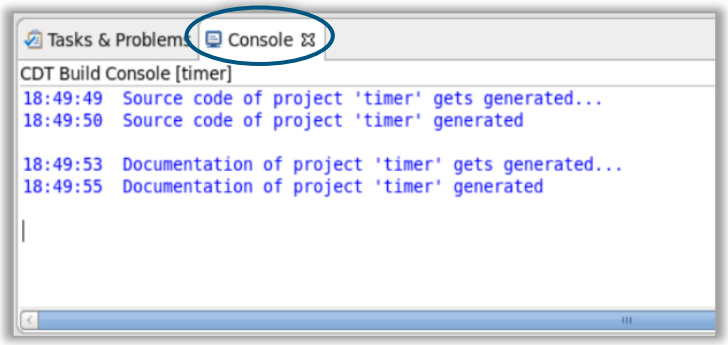
    Click ***Build*** button to build using the FastBuild configuration. Click the arrow to view more options. Now, you can configure the following on top of the same build settings by using any of the following options.

    - Select the '***Debug Build Settings***' option if you want `Debug` setting to be always `on` and the `Optimize` to be `off`.

    - Select the '***Release Build Settings***' option if you want `Debug` setting to be always `off` and the `Optimize` to be `on`.

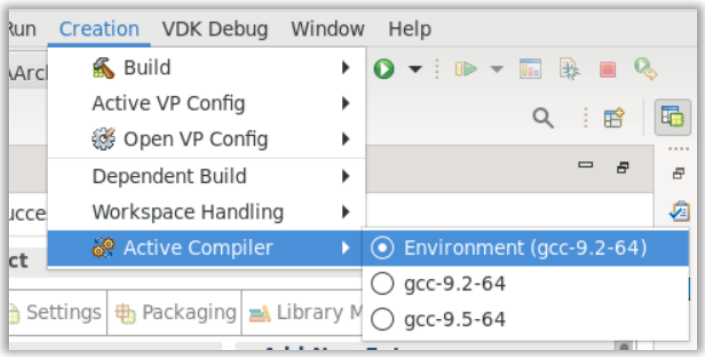
    Note: The build process also saves the project (if not saved) and generates the code (if it needs to be generated).

Click the ***Generate Documentation*** option to generate related model documentation. The documentation build is separate to save the source code generation build in case you don't need the documentation update. Use this option as and when required.



By default, the unit test code gets generated and unit test is built. All the generated files are saved under `<workspace>/<project name>` i.e. `<workspace>/timer` directory. If you use the ***Save As…*** option to save the project as `timer.tlmc` in some other location, the resultant code and other files get generated in the same path accordingly (`BASE_DIR`).

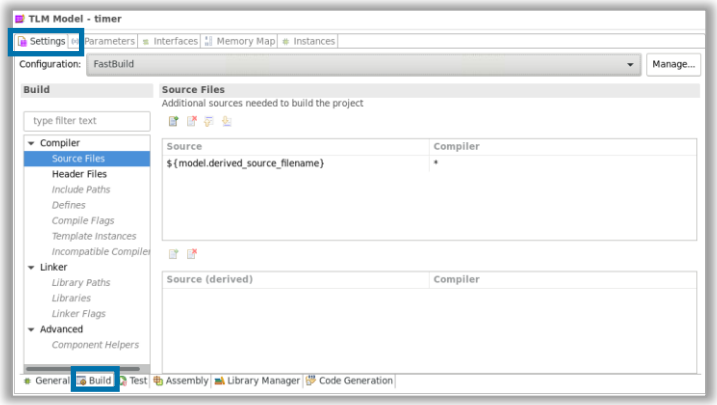
For setting the active compiler version to be used for the build, click ***Active Compiler*** option from the ***Creation*** menu, and select from the available list of compilers, or let the environment (setup) setting also select the compiler.

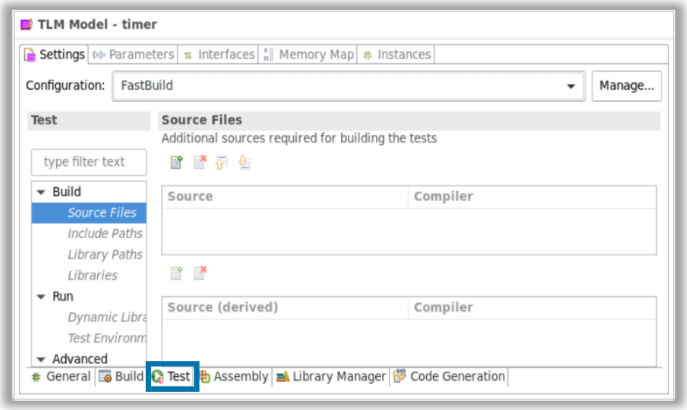


You can view the generated files in the ***Project Explorer*** and monitor the build messages in the Console.

Configuring Additional Build and Test Options

- Click to open the ***Build*** and ***Test*** page to examine the available configurations settings.





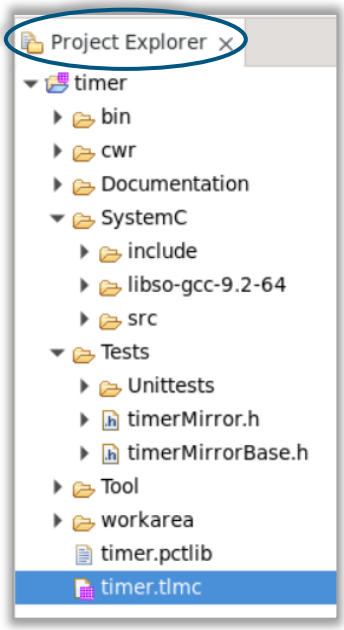
The library type is set to be dynamic (`FastBuild`) or static. You can also use some other settings such as include paths, defines, compile flags, source files, template instances, libraries, library paths, linker flags. Note that these can be compiler specific settings.

For this lab exercise, we do not intend to change anything in the build configurations.

Task-9: Examine the Generated Infrastructure and Files

- *Motivation: When you build your model project, Virtualizer Studio generates the source code, documentation, unit test, and other files related to our model. In this step, we will take a closer look at the generated output to understand the model creation flow better.*

To view the generated files, switch to the ***Project Explorer*** view, expand the `timer` model project folder and review its content:



- `SystemC`: Contains the generated code for the model (timer) component.
  - Header files in `include` directory
  - Source files in `src` directory
  - Libraries in `libso-<compiler>` directories
- `Tests`: Contains the generated test framework.
- `Documentation`: Contains the generated model documentation. We need to build the documentation explicitly using the ***Generate Documentation*** option from the Build menu.

To open any file, double-click on the file name in the ***Project Explorer***.

The generated XML model (for VDK Creator or Platform Creator library) can be found under `<workspace>/<projectname>/Tool/PCT` directory as `<projectname>.xml` file.

Let's now look at the generated model code.

Review Generated Code

Let's now review the following source code files:

- `timerBase.h`
- `timer.h`
- `timer.cc`

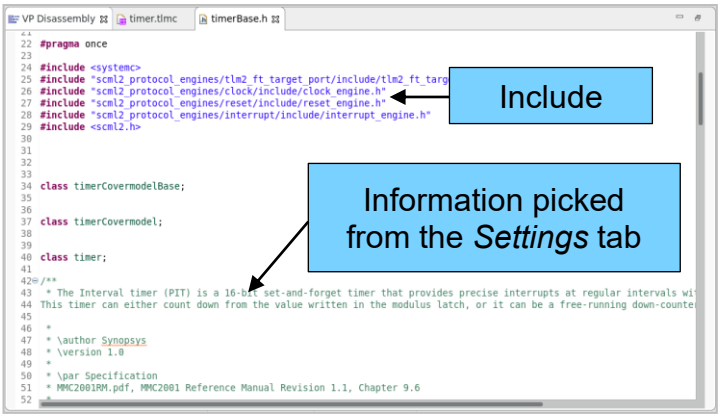
Exploring `timerBase.h` File

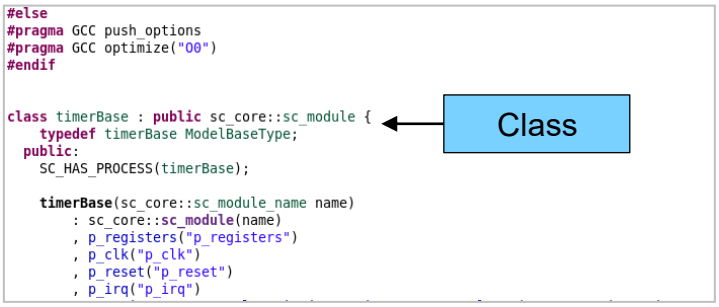
18. To open the file, double-click on the `timerBase.h` file from the `timer/SystemC/include` location.

Note: DO NOT edit this file as it gets overwritten every time the model is exported.

This base file has the declarations for all the objects that you have defined using any tab of the Model Editor window such as:

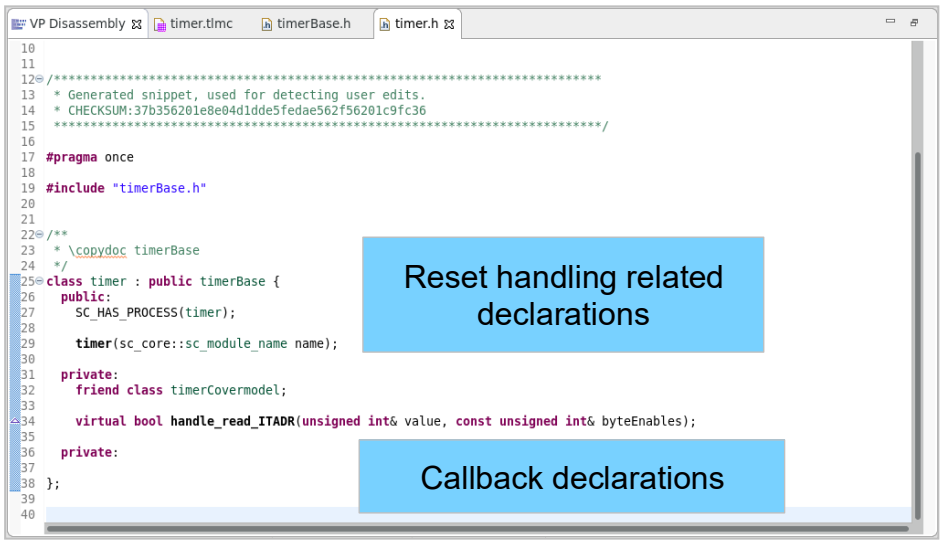
- Interfaces
- Registers
- Register callback routines
- Callbacks declarations
- Function to reset model, and so on





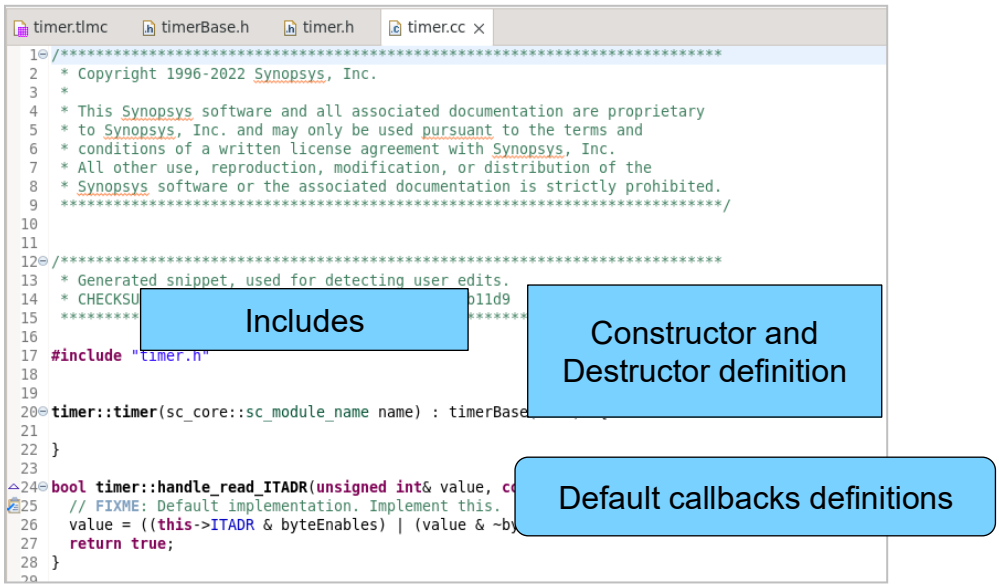
Exploring timer.h File

Note: This is the file carrying the user defined behavior definition - it will not be overwritten. You can edit it directly but the recommended way is to use e.g. the TLM Creator pages and peep-hole views for callbacks and behaviors.



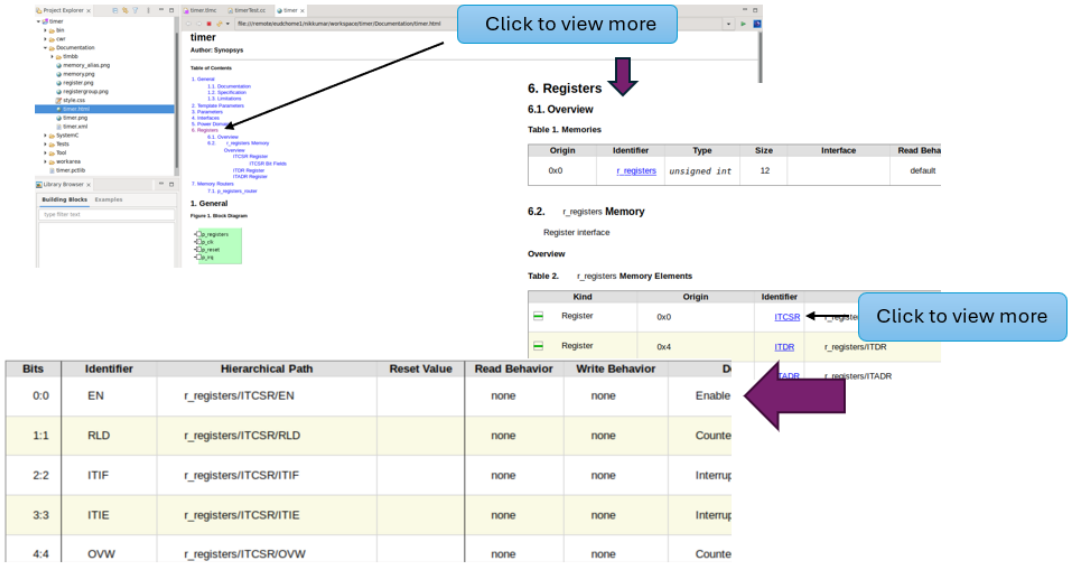
Exploring timer.cc File

Note: This is the file carrying the user defined behavior definition - it will not be overwritten. You can edit it directly but recommended is to use e.g. the TLM Creator pages and peep-hole views for callbacks and behaviors.



Review Generated Model Documentation

Virtualizer Studio also generates nicely structured html documentation for your model. It can be viewed in the Virtualizer Studio as well as in any HTML browser. When you use the model in VDK Creation technology of Virtualizer Studio (or you import this model to Platform Creator), this serves as the online model documentation. The HTML document opens in the Virtualizer Studio environment for the ease of reviewing.



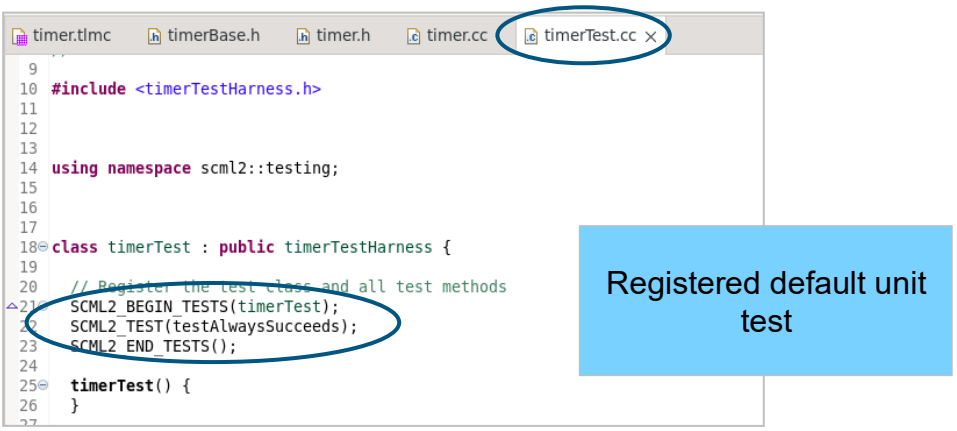
In Virtualizer Studio's Project Explorer, expand ***Documentation***, and double-click `timer.html` file as follows.

Task-10: Build and Run Default Unit Tests

- *Motivation: The TLM Creator technology also generates a `unittest` class and debug infrastructure for your model in the `Tests/Unittests` directory. In this step, we will run the default unit tests and explore the results.*

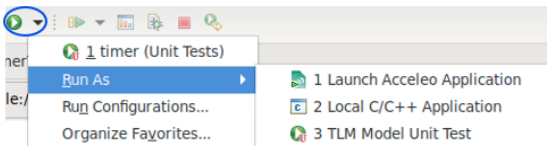
The basic set of unit tests are defined in the `timerTest.cc` C++ source. When we start implementing the timer behavior, we will be adding the unit tests to the same file. Lets explore on the default implementation.

Double-click on the timerTest`.cc` file from the Project Explorer to open it in the editor.



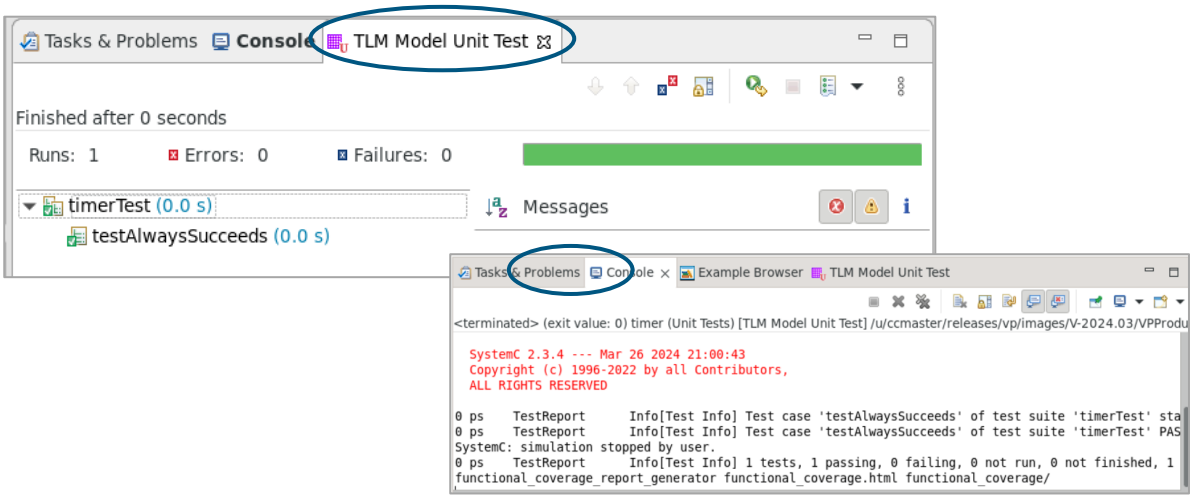
The default unit test serves mainly as a starting point for developing your own unit tests. You will learn how to extend the unit tests in the subsequent Labs.

For this exercise, we will cover the steps to compile and run the default unit test. The main purpose is to verify that the generated code is consistent and can be compiled without any errors before you start adding your own code.

- 

Select the `timer` project in the ***Project Explorer*** and run the unit tests as follows.
  - Click the ***Run*** toolbar button and select ***Run As > TLM Model Unit Test*** option.

Check the results in the ***TLM Model Unit Test*** tab and output messages in the ***Console*** view.



Task-11: What We Have Learned

In this module, we performed the following tasks:

- Create/use a Tracking Feature List for the `timer` model.
- Create a new TLM Creator Project from an empty template model.
- Add model elements such as its registers and interfaces.
    - Optional flow: import the memory elements from an IP-XACT specification.
- Generate the code for the model and build it.
- Examine the generated infrastructure and code.
- Build and run the default Unit Test as sanity test.

Task-12: What's Next

In the next lab, we will implement the behavior of the timer.